image but under a certain price". A database planner would first run a vector search operator on the vector attribute of image embedding to find K nearest tuples, then apply a filter operator on the price attribute. But it is impossible to predict exactly how many tuples will pass the filter operator, which could be much less than K. Therefore this practice has the inherent difficulty of identifying the optimal K that produces exact X results. As a result, it resorts to either a setting of a conservatively large K or a trial-and-error of many Ks, which both lead to suboptimal query performance.

In this paper, we present VBASE, a new system capable of efficiently serving complex online queries that involve both approximate similarity search and relational operators on scalar and vector data-sets. VBASE identifies *Relaxed Monotonicity* as the common property abstracted from the two seemingly different systems: vector search systems and relational databases. *Relaxed monotonicity* requires index traversals to only follows monotonicity *approximately*. We observe that state-of-the-art vector indices all follow relaxed monotonicity property in a two-phase pattern: an index traversal first locates the nearest region to a target vector approximately, and then moves away from the target region progressively in an approximate way. Based on the observation, we formally define *Relaxed Monotonicity* property, which abstracts the core index traversal pattern that most existing vector indices already have (§3.1). *Relaxed monotonicity* can be viewed as a generalized form of monotonicity, thus it is also applicable to conventional scalar indices, such as B-trees. Therefore, *Relaxed Monotonicity* can serve as the common foundation of vector search and database systems.

With relaxed monotonicity, VBASE builds a unified query execution engine to support a wide range of queries both on scalar and vector data, including queries across multiple heterogeneous indices. VBASE's unified engine is based on a `Next` interface, instead of `TopK`, to support traversal in both vector and scalar indices. Meanwhile, the engine allows the derivation of a generalized termination condition from relaxed monotonicity to stop a query's execution timely.

A unique characteristic of VBASE is that its relaxed-monotonicity-based query execution engine can *provably* achieve *equivalent* query results to those produced by `TopK`-only solutions of the *optimal* $\widetilde{K}$ (§3.3) This powerful property allows VBASE to circumvent the constraints of a `TopK`-only interface to achieve significantly higher efficiency, while preserving the semantics of `TopK`-based queries. In particular, based on the derived generalized termination condition, VBASE is able to detect the $\widetilde{K}$ during index traversal without an prediction of $\widetilde{K}$. This allows VBASE to achieve similar performance to the well-optimized `TopK` vector search. For more complex queries than `TopK` searches, VBASE can achieve up to three order-of-magnitude lower average and tail query latency over state-of-the-art systems under similar result accuracy (i.e., same or even better recalls).

Moreover, with relaxed monotonicity, VBASE can even support approximate query types that previous systems do not, and show superior query performance and accuracy. For example, VBASE can finish a `join`-based vector query in 16 seconds with 99.9%+ recall rate, which is 7000× faster than a brute-force table scan.

Table 1: Online Similarity Query Support for Vectors

	S1	S2	S3	S4
ANN systems [25,43,46]	✓	✗	✗	✗
AnalyticDB-V [80]	✓	✓	✗*	✗*
PASE [86]	✓	✓	✗*	✗*
PostgreSQL [12]	✗*	✗*	✗*	✗*
Milvus [76]	✓	✓	✓	✗
Elasticsearch [4]	✓	✓	✗◇	✗

∗: Some systems can support these queries through exhaustive linear scan, but this cannot meet the requirements of online services.
◇: Only support one inverted index and one vector index.

In summary, we make the following contributions:

1. VBASE identifies and defines formally "*Relaxed Monotonicity*", a property that reveals, for the first time, the core of well-designed vector indices and why they work effectively in practice.
2. VBASE builds a *Unified Database Engine* based on *relaxed monotonicity*, which enables powerful complex queries leveraging both vector and scalar data indices.
3. We prove that VBASE's unified engine produces equivalent results to `TopK`-only methods using vector indices, with a much more efficient execution plan than that of `TopK`-only methods.
4. We implement VBASE based on PostgreSQL with 2000 lines of additional code, and show an end-to-end evaluation of eight complex SQL queries on a hybrid one million recipe data-sets [59] with both vector and scalar attributes.

We plan to make VBASE open-source to satisfy the emerging important vector analytic applications in the era of AI.

2 Background

2.1 Emerging Online Vector Queries

Vector has become a key form of data representation in the AI era. Deep learning has enabled a growing number of vector-centric online applications, including embedding-based retrieval [25,87], face recognition [69], code retrieval [37], question answering [52,63], Google Multisearch [7], Facebook near-exact duplicates detection [6], etc. More recently, AI applications have leveraged ChatGPT's retrieval plugin [10] to convert their proprietary knowledge, personal documents, and chat contexts into vectors. This enables the retrieval of relevant vectors with price, category, location, or time constraints to construct prompts in the chat.

Traditional applications also benefit from vectors empowered by AI. For example, search engine turns web documents into both bag-of-words sparse vectors and deep learning em-

bedding dense vectors to improve the relevance of search results. And recommendation systems turn images, videos, and descriptions of items into different vectors. Combined with scalar data like item price and category, these vectors are used to enhance recommendation experiences.

All these call for a general system to run sophisticated vector and scalar queries efficiently. In summary, these vector scenarios can be categorized into the following types.

S1: Single-vector `TopK`. embedding-based retrieval [25], recommendation [87], and question answering [52, 63] essentially search a vector data-set for the K closest vectors, given a query vector. Such queries can be naturally expressed by a `TopK` operator on a single-vector column.

S2: Single-vector `TopK` plus scalar attribute filtering. There are also requirements to find `TopK` results under certain scalar attribute constraints. Google Multisearch [7] belongs to this category. It allows users to provide additional text hints during a similarity image search.

S3: Multi-column `TopK`. Some vector analytics require intersecting results of multiple `TopK` searches over different vector attributes. For example, image-recipe retrieval [68] is a recipe search on multi-modal data attributes of both (vectorized) ingredient keywords and a sample dish image. Recent work [70, 83] shows that multi-column `TopK` search can boost result quality in applications such as question-answering.

S4: Vector similarity filter. Similarity filtering is a typical vector analytics scenario. For example, face recognition [69] and Facebook's near-exact duplicates detection [6] search for similar images (given an image) from a data-set with a similarity threshold. To support such applications, one could use vector filtering based on distance similarity between two images, i.e., distance-based range query.

All these vector query types have a strict latency requirement (e.g. milliseconds). Unfortunately, no existing systems can support all these online similarity queries comprehensively and efficiently (see Table 1).

2.2 The Division Between Databases and Vector Search Systems

Although databases can express the above queries through relational algebra, the division in the semantics between vector and conventional database indices makes it difficult to provide a unified system that efficiently runs various types of sophisticated online vector queries as shown in Table 1.

Relational database. A relational database is one of the most prominent tools to run sophisticated queries [16, 24, 29, 40]. In order to meet the low-latency "online" requirement, indices are widely adopted by databases to expedite query executions, such as B-tree [22], B+-tree [75] and more. These indices demonstrate *monotonicity*, a property that allows a query to traverse an index monotonically along a certain direction, e.g., in a descending or ascending order.

One of the most important types of online queries in the context of emerging vector scenarios is `TopK` query (§2.1). And a conventional database index can speed up `TopK` by traversing the index in the ascending or descending order and terminating the query as soon as it collects K results. This optimization applies to many `TopK` variants, such as `TopK` + filtering, and multiple-column `TopK` queries [33].

However, the effectiveness of such optimization relies on the assumption of monotonicity, which high-dimensional vector indices do not follow. We elaborate next.

Approximate vector search. The recent eruption of AI models has been generating a large and growing amount of high-dimensional vector data. For better learning representation, a vector can have hundreds of dimensions [60, 66, 88]. Due to the curse of dimensionality [28], no solutions can complete a high-dimensional vector query in sub-linear time. To address "online" scenarios, modern vector search systems resort to approximation to lower query latency dramatically (milliseconds) while maintaining a relatively high result accuracy (90%+ recall). These systems are often referred as *approximate nearest neighbor search* (ANNS) systems [5, 25, 43, 55, 57, 85].

Like relational databases, vector indices are adopted to facilitate approximate vector search. Representative vector indices are either organized as *partitions* (clustering-based [5, 17, 19, 25, 44, 45, 48, 90], hash-based [30, 41, 79, 81, 85]), high-dimensional tree-based [23, 57, 62, 78]), or *neighborhood graphs* [32, 39, 43, 55, 58, 77]. The difficulty of locating a vector in the high-dimensional space forces these vector indices to optimize for approximate `TopK`. In a `TopK` query, index traversal is guided by a query vector q approximately towards the nearest neighbors tortuously based on the distance between q and some anchor points (e.g. cluster centroids, or sampled graph vertices). During the traversal, the direction to q may change dramatically, thus the process does not guarantee to approach q in every traversal step and the vector index traversal is not monotonic.

The lack of monotonicity in vector indices bars database systems from directly using vector indices to expedite queries, which is the primary source of *the division between databases and vector search systems*.

`TopK`-based solutions to eliminate the division. Because vector indices are optimized for `TopK`, ANNS systems expose only a `TopK` interface. To close the monotonicity gap between databases and vector search systems, the current practice is to use ANNS `TopK` interface to create tentative indices based on K vectors sorted according to the distance to the target vector. Such tentative indices preserve monotonicity, which enables fast vector query processing in databases [76, 80, 86].

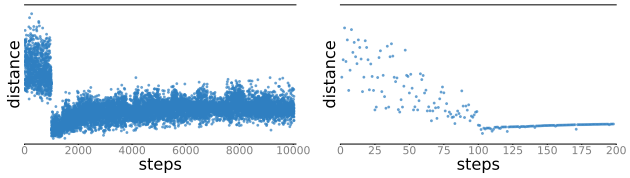
However, `TopK`-based tentative index solutions are unsatisfactory. It is difficult, if not impossible, to predictthe right size of $\widetilde{K}$ for the tentative index for queries, where a subsequent relational operator with a filter constraint can collect just the right number of results. This limitation universally applies

to TopK + filter queries, vector similarity filter queries, and more. Thus TopK-based tentative indices inevitably lead to choosing a conservatively very large K [80, 86] or performing trial-and-error with different sizes of K [76], both incurring excessive data accesses and computations.

3 VBase Design

3.1 Relaxed Monotonicity

Unlike conventional scalar indices, high-dimensional vector indices are designed for approximate TopK and do not follow monotonicity. Figure 1 shows the TopK traversal patterns of two popular vector indices, FAISS IVVFlat [5] and HNSW [89] . As illustrated, vector index traversal does not comply with monotonicity, the distance towards the target vector oscillates *unpredictably* as the traversal progresses. A lack of monotonicity in these vector indices bars relational databases from directly using them to expedite queries (§2.2).



(a) FAISS IVFFlat (b) HNSW

Figure 1: Traversal patterns of two vector indices.

The Two-phase Vector Index Traversal Pattern. Nevertheless, Figure 1 reveals a two-phase index traversal pattern for both vector indices. In the first phase, the index traversal approaches the target vector region approximately in spite of large oscillations in vector distances. In the second phase, the index traversal stabilizes and steadily departs from the target vector region in an approximate way.

This two-phase traversal pattern is common in most vector indices we examine. We believe that the essence of well-designed vector indices is an effective data-structure that embodies this traversal pattern implicitly. Thus a TopK search query could terminate early when it enters the second phase as further traversals are unlikely to identify more similar vectors.

The Formal Definition of Relaxed Monotonicity. Based on the two-phase traversal pattern, we can formally define *Relaxed Monotonicity* which identifies if a vector index traversal has entered the second phase. The definition is built upon the intuition of how a vector TopK search is executed internally.

Figure 2 shows such an intuition by illustrating the process of a general vector search for a query vector q. The dashed arrow in the figure shows an index traversal path with respect to q. Following the two-phase pattern, the query first approaches the *neighborhood* of q gradually. In a high-dimensional space, the neighborhood of q is defined by a neighbor sphere centered around q, illustrated as a circle in Figure 2. Afterward,

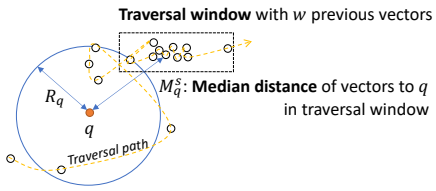


Figure 2: An Illustration of *Relaxed Monotonicity*'s intuition. A vector query q discovers q's neighborhood with E nearest vectors along the progression of a traversal path.

the index traversal leaves the sphere and enters phase two, where the query can terminate in this phase.

Figure 2 suggests that, to determine whether it enters phase two, a query needs to understand R_q, the radius of the neighbor sphere centered around q, and whether M_q^s, the distance between the query's current index traversal position (denoted as traversal step s) and q, is greater than R_q, i.e., it is traversing beyond the neighborhood of q.

Formally, R_q, the radius of q's neighborhood, is defined as:

$$R_q = Max(TopE(\{Distance(q, v_j) | j \in [1, s-1]\})), \quad (1)$$

where $TopE$ denotes the E nearest neighbors of q observed during the traversal, supposing that the traversal has reached step s so far. For a K nearest vector search query, it requires $E \geq K$ in order to produce sufficient final results. In Figure 2, the E vectors within the circle are the nearest neighbors of q of all the s vectors visited so far. During an index traversal, the sphere's radius R_q would gradually decrease during phase 1, and becomes stable during phase 2.

Given the definition of R_q, the system needs to define M_q^s, the distance measurement between the target vector q and the current index traversal position, denoted as traversal step s. M_q^s is then used to determine whether the traversal enters phase 2, i.e., leaving the neighbor sphere.

Mathematically, M_q^s is defined as the median distance to q of all vectors traversed in the most recent w steps, i.e., the traversal window.

$$M_q^s = Median(\{Distance(q, v_i) | i \in [s-w+1, s]\}), \quad (2)$$

where $Distance(q, v_i)$ denotes the distance between q and vector v_i, traversed in the past traversal window. Note that we use *median* instead of *mean* to disregard any outlier vectors in the traversal window, which has exceedingly large or small distances to q than others. For example, the two outlier vectors in the leftmost and rightmost positions in the traversal window shown in Figure 2.

Taking Eq.1 and Eq.2 together, we define *Relaxed Monotonicity* as:

Definition 1 Relaxed Monotonicity

$$\exists s, M_q^t \geq R_q, \forall t \geq s. \quad (3)$$

In other words, Def. 1 determines that a vector index follows *relaxed monotonicity* if there exists a certain index traversal step s, where all traversal steps t after step s transcends into a region (M_s^t as the region's distance to q) that is outside q's neighborhood sphere, defined by q's E nearest neighbors.

Importance of Relaxed Monotonicity. Relaxed monotonicity is the key for the database to circumvent the inefficient constraint of `TopK`-only interfaces, and to generate an efficient execution with on-the-fly early termination. When subsequent database operators following the vector index scan can determine that vector index traversal has entered the second phase, one would know that we are veering away from the target vector steadily. In such cases, we could early terminate the query if sufficient results have been collected, because new tuples with closer vector attributes are unlikely to be found.

Generality of Relaxed Monotonicity. All mainstream vector indices listed in ANN Benchmarks [2] perform vector search using four general components: 1) *index traversal* to navigate the vector data-set; 2) *termination check* to detect query termination signal; 3) *monotonicity check* to determine if a query enters Phase 2; and 4) *priority queue* for keeping K nearest vectors so far. Often in ANNS indices, *monotonicity check* is a necessary condition for the *termination check*.

Although vector indices implement these four components in different ways, their monotonicity check satisfies Def. 1. For example, Figure 1 shows that index traversal patterns of IVFFlat and HNSW follow *relaxed monotonicity* obviously. And Def. 1's parameters, i.e., the traversal window w and the neighborhood of q R_q, are able to capture the internal characteristics of index traversal patterns of these popular vector indices as well as conventional indices. Next, we elaborate on the setting of these parameters for representative indices.

- **Graph-based Vector Indices**, such as HNSW [89], follow the two-phase pattern using graph data-structures. In the first phase in Figure 1b, HNSW quickly navigates the traversal to the neighborhood of q through hierarchical coarse-grained to fine-grained navigating graphs. When it reaches the fine-grained graph, the traversal enters the second phase where it has found the neighborhood of q and departs away. Vector search using a graph-based index use best-first (BF) graph traversal from a fixed starting point. BF search maintains a *sorted candidate queue* with the size ef. This queue essentially represents the neighborhood sphere in Eq. 1 with ef vectors. Therefore E equals ef for HNSW. BF traversal explores the graph through the vectors in the candidate queue and expands the exploration by visiting their neighbors. If a neighbor is *unvisited* and its distance to the target vector q is smaller than the farthest vector in the sorted queue (i.e. R_q), the traversal replaces the farthest vector with the new one and resorts the queue. Because the traversal only compares the traversed vector itself with R_q, the traversal window w in Eq. 2 equals one.
- **Partition-based Vector Indices**, such as FAISS IVFFlat [5] and SPANN [25], divide vectors into multiple clusters where nearby vectors are conglomerated. During the traversal in the first phase in Figure 1a, IVFFlat traverses over the centroids to identify the m closest clusters, and then in the second phase it goes over the vectors in m clusters and terminates when all vectors in m clusters are traversed, which indicates Eq. 3 of *Relaxed Monotonicity* has been satisfied after the vector search visits m clusters. With this observation, E in Eq. 1 is set to K of the `TopK` query, and the traversal window w in Eq. 2 is set to the number of total vectors in m clusters.
- **Scalar Indices**, such as B-Tree, follow strict monotonicity. It is a special case of relaxed monotonicity, where w and E are both set to 1 and Eq.3 is always true.

Note that it is our observation that well-designed indices should satisfy *Relaxed Monotonicity*. VBASE abstracts and formalizes this property that most indices already preserve, and encapsulates it with mathematical terms such as E and w in Eq. 1 and 2. It is the responsibility of individual indices to guarantee relaxed monotonicity by tuning the corresponding hyperparameters like ef and m, which can be transformed to E and w, as discussed above.

3.2 Unified Query Execution Engine

With relaxed monotonicity, VBASE builds a unified query execution engine modeled after a traditional database engine with minimum changes. VBASE's unified engine is built on Volcano Model (i.e. Iterator Model) [35], where 1) a relational operator in a given query produces a stream of tuples iteratively that are consumed by downstream operators, and 2) the iterative execution stops if a termination condition is met.

Iterative Execution Model. VBASE fully complies with the Volcano Model. It reuses the traditional `Open`, `Next`, and `Close` interfaces to leverage index traversal so that no change is required for conventional indices. For vector indices that traditionally expose `TopK` interfaces only, VBASE performs a simple adaptation to expose their internal index traversal process, to conform to VBASE's `Next` interface. We will discuss the details of implementing `Next` for vector indices in §4.2.

Generalized Termination Condition Check. VBASE modifies the termination condition based on relaxed monotonicity. In particular, VBASE extends the original termination condition with *relaxed monotonicity check*. In addition to the original query termination condition, VBASE performs relaxed monotonicity check by inspecting Eq. 3. Because VBASE requires a vector index guarantees *Relaxed Monotonicity*, an inspection of Eq. 3 could be reduced to $M_q^s > R_q$, where *relaxed monotonicity* checks beyond step s are all assumed to be true.

The query execution would only stop if both the original termination condition and *relaxed monotonicity* check were passed. Please note for the traditional index, the *relaxed monotonicity* check is always true, which reduces to the termination check of the convention iterative model.

Next, we describe how VBASE's termination conditions work for TopK and distance-based ranger filter, two operators used by vector queries.

- **OrderBy with limit:** In traditional databases, TopK is usually expressed by an OrderBy operator with limit K. The traditional TopK query terminates immediately once K results have been collected, because all indexed tuples are ordered. For an approximate TopK query on vectors, VBASE need to check if the relaxed monotonicity check (i.e., reduced Eq.3) is passed, in addition to collecting K vectors. When *relaxed monotonicity* check is true, the index traversal has entered phase 2 (§3.1), which indicates it is veering away from the target vector steadily. And we can terminate the query after collecting K nearest vectors, because it is unlikely to find new vectors closer than the collected ones.
- **Range filter:** Distance-based range filter returns tuples whose values or distances to a target are within a range R. For a conventional query, the query stops when a tuple being traversed goes beyond range R, because monotonicity guarantees we have visited all tuples within R. For a vector query, the execution only stops if both a vector along the traversal path exceeds distance R and the relaxed monotonicity check is passed, which indicates we are in a stable phase (phase two) moving away from a target vector.

Advantages of a Unified Engine. The unified engine enables VBASE to preserve full compatibility with traditional databases and supports all the vector query types discussed in §2.1. It also creates new optimization opportunities for vector queries. For example, instead of filtering after TopK, VBASE can perform filtering during index traversal with flexible termination conditions (for TopK or range query). This optimization is one of the key reasons VBASE outperforms TopK-based solutions (§5.3). Moreover, the unified engine allows the incorporation of a refined NRA algorithm [33], which can significantly improve the performance of multi-column vector query (§4.4).

Interestingly, VBASE's unified engine also supports vector Join. Although not an online query, vector Join is useful in scenarios such as document auto-tagging [26, 72], where a small labeled (tagged) document set is used to identify a tag for each document in a large unlabeled document set by finding the document pair with the closest document embeddings (vectors). This can be thought of as running a Join operator on a document table and a label table with a distance-based match, which can be achieved in VBASE by an index join based on a range filter. Because such a Join can only be achieved by a full table scan in existing solutions, VBASE can outperform the baseline by over 7000× (§5.3).

3.3 Result Equivalence

In this section we demonstrate a powerful property of VBASE's unified query execution engine based on *Relaxed Monotonicity*, that it produces the equivalent results as a TopK method based on a tentative monotonicity-preserving index with the optimal $\widetilde{K}$. The optimal $\widetilde{K}$ is the minimal K' for the index traversal to satisfy K results in a TopK query. As K' is the minimal satisfactory value, the query latency is minimized. We rely on the quality of individual indices to ensure recalls.

Next, we formally prove the result equivalence for TopK+filter and range filer [20] queries, which are major types of similarity searches supported by existing TopK-based vector systems (Table 1). The proof also reveals the reason of VBASE's superior performance.

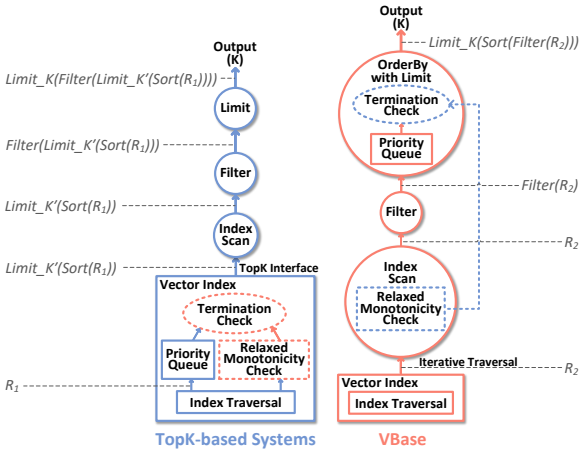


Figure 3: Result Equivalence

TopK +filter. To find K vectors matching the filter, a TopK-based system first collects K' vectors by calling TopK (K') and sorts the collected K' vectors. To achieve this, the system needs to traverse R_1 vectors through the underlying vector index. The query then runs on the sorted K' vectors by applying the filter and the K limit operators, producing the final results, denoted as r_1. Eq.4 formulates the above process, illustrated on the left in Figure 3.

$$r_1 = Limit\_K(Filter(Limit\_K'(Sort(R_1)))). \tag{4}$$

We define $filter\_selectivity$ as the ratio of the output set on the input set of the filter operator. Eq. 4 can then be transformed to:

$$\begin{aligned} r_1 =& Limit\_K''(Filter(Sort(R_1))), \\ & where\ K'' = min(K, K' \times filter\_selectivity). \end{aligned} \tag{5}$$

Assuming the TopK-based system can predict the optimal $\widetilde{K}$, i.e., $K' = \widetilde{K} = K/filter\_selectivity$, execution will get exactly K results, and Eq. 5 reduces to Eq. 6

$$r_1 = Limit\_K(Filter(Sort(R_1))). \tag{6}$$

In comparison, VBASE traverses R_2 vectors via the same vector index as the TopK-based system, and gets results r_2. Eq. 7 formulates this process, shown on the right of Figure 3.

$$r_2 = Limit\_K(Sort(Filter(R_2))). \tag{7}$$

We show in §3.2 and §4 that TopK-based systems and VBASE use the same vector index, follow the same index traversal algorithm, and are based on the same relaxed monotonicity check to terminate queries. Therefore both systems traverse the exact same set of tuples, i.e., $R_1 = R_2$. [1]. Because $R_1 = R_2$ and $Filter$ and $Sort$ are commutative, from Eq. 6 and 7 we conclude $r_1 = r_2$. **Q.E.D.**

It is difficult for a TopK-based solution to predict K' to get both correct results and high query efficiency. If $K' \times filter\_selectivity < K$ in Eq.5, the system cannot get enough results, leading to poor accuracy. If $K' \times filter\_selectivity > K$ in Eq. 5, the result is accurate at the expense of splurging extra index traversal. Some TopK-based system [76] performs trial-and-error with many values of K' until $K' \times filter\_selectivity \geq K$, which results in excessive duplicated data access and processing. In contrast, VBASE determines $\widetilde{K} \times filter\_selectivity = K$ on-the-fly, therefore achieving both high query accuracy and performance.

Range Filter Query. A range filter query based on TopK can be formulated as

$$r_1 = Filter(Limit\_K'(Sort(R_1))), \tag{8}$$

Where R_1 stands for the traversed vectors by the underlying TopK primitive. Similar to Eq.4 vs. Eq.5, Eq. 8 can be transformed to

$$\begin{aligned} r_1 =& Limit\_K''(Filter((Sort(R_1)))), \\ & where\ K'' = K' \times filter\_selectivity. \end{aligned} \tag{9}$$

Under the assumption of optimal $\widetilde{K}$, assuming the query produces T vectors, we have $\widetilde{K} = K' = T/filter\_selectivity$. Based on $\widetilde{K}$, Therefore,

$$r_1 = Limit\_T(Filter(Sort(R_1))) = Filter(Sort(R_1)). \tag{10}$$

In comparison, VBASE traverses R_2 vectors via the same index and gets results r_2, which can be formulated as

$$r_2 = Filter(R_2). \tag{11}$$

Since the traversal algorithm and the termination condition are exactly the same as the TopK-based solution, both systems visit the same set of vectors, i.e., $R_1 = R_2$. As the filter conditions are not sensitive to order, $r_1 = r_2$. **Q.E.D.**

4 VBASE Implementation

4.1 Relaxed Monotonicity Check

We implement a common relaxed monotonicity check for all vector indices based on Definition 1 in §3.1. Specifically, we implement two queues to track the current traversal state: 1) a priority queue with size E called smallestQueue, to keep the visited nearest neighbors of a target vector q during the traversal; 2) recentQueue of size w to track the most recent traversal window. When a new vector v is visited via index traversal, smallestQueue and recentQueue are updated accordingly. And the relaxed monotonicity check is performed by calculating the current traversal state according to Eq.(3) based on vectors in smallestQueue and recentQueue.

Note that E and w are sensitive to data distribution and specific indexing algorithms. Increasing them tends to improve query accuracy at the expense of longer latency. In practice, they can be tuned to trade off query accuracy and latency.

4.2 Query Execution Engine

VBASE's unified query engine is implemented based on PostgreSQL, with minor extensions to modules regarding index traversal and termination conditions.

Vector index integration. Existing high-dimensional vector indices only expose TopK interface and keep the index traversal and relaxed monotonicity check internally to the system. VBASE re-architects the vector indices systems by exposing the internal index traversal algorithms with Open, Next, and Close interfaces, which can then be integrated into Volcano Model seamlessly.

VBASE has incorporated several state-of-the-art vector indices, including HNSW [89], IVFFlat [5] and SPANN [25], where SPANN is shown effective for billion-scale vector datasets. Next, we introduce the integration of HNSW and IVFFlat, a graph-based index and a partition-based index, respectively. Other algorithms can be integrated in a similar way.

HNSW [89] is a graph-based vector index consisting of hierarchical neighborhood graphs where the upper-layer graph keeps coarse-grained samples of the lower-layer graph. A query traverses the graphs from upper-layer to lower-layer following the best-first manner. The approximate nearest point found in the upper-layer graph is the entry point of the lower-layer graph. In VBASE, we remove the implementation regarding TopK, e.g., a priority queue to record the top k results, and only keep states necessary to carry on the index traversal algorithm. The relevant states include a bitmap to record previously visited vectors, the current vector being visited, and the candidate vectors to be visited next. These states will be kept during the query life cycle. To initiate a query on a vector index, VBASE calls Open to search on high-layer graphs. During query execution, each call to Next will return the current closest unvisited node, records it, and expands its neighbors into candidate vectors in the state. The state will be cleared in Close function. Overall, we modify less than 200 lines of code to integrate HNSW.

IVFFlat [5] is a partition-based index, which clusters vectors into lists and chooses the centroid as the representative of each list. In the Open interface used to initiate index traversal, VBASE sorts all the lists from near to far based on the dis-

[1] We assume vector index traversal is deterministic, true for most vector search systems.